planarRefiner for OpenFOAM Foundation 14

Reusable 2-D Slab and Wedge/Axisymmetric Adaptive Mesh Refinement

Version 1.0.0 - September 2026

Component identity. `planarRefiner` is a runtime-selectable OpenFOAM 14 **`fvMeshTopoChanger`**, built as `libplanarFvMeshTopoChangers.so`. It is not an `fvModel` and does not depend on detonationFoam. It was developed inside the OF14 detonationFoam migration, but its portability was qualified with stock `incompressibleFluid`.

Table of Contents

Table of Contents (continued)

1. Purpose and scope

OpenFOAM 14 has strong native 3-D dynamic refinement support. The purpose of `planarRefiner` is narrower: provide a reusable dynamic-refinement path for **true two-dimensional** meshes where refinement must occur only in the two active geometric directions.

The release supports:

- planar slab meshes using `empty` constraint patches;
- wedge/axisymmetric meshes using paired `wedge` patches;
- uniform or localized scalar-field-band refinement;
- MPI redistribution callbacks;
- native OpenFOAM 14 runtime load balancing;
- restart from an already refined/distributed mesh.

The current release is **refinement-only**. Automatic unrefinement/coarsening is not implemented.

2. Why topology change belongs in `fvMeshTopoChanger`

2.1 `fvModel` versus `fvMeshTopoChanger`

An OpenFOAM `fvModel` primarily contributes source terms to finite-volume equations. It can receive mesh-change callbacks so its own data survives topology changes, but it is not the normal owner of dynamic topology operations.

An `fvMeshTopoChanger` is the runtime-selected mesh component that participates in the standard `mesh.update()` topology-change path. Therefore `planarRefiner` is implemented as:

```
class planarRefiner : public fvMeshTopoChanger
```

and registered with:

```
addToRunTimeSelectionTable(fvMeshTopoChanger, planarRefiner, fvMesh);
```

This is why the component can be used by multiple solvers without embedding AMR code inside each solver.

3. Refinement theory

3.1 Cell selection: $L \le \phi_c \le U$

The user selects a registered `volScalarField` and a lower/upper band. Cell c is selected if

$$L \le \phi_c \le U .$$

For a detonation case, the qualified example used temperature:

$$800\,K \le T_c \le 4900\,K ,$$

which selected the shock/reaction transition rather than refining the entire domain.

3.2 Directional planar split: $N_{new} = N_{old} + 3\,N_{sel}$

A true 2-D mesh has one inactive geometric direction. `planarRefiner` queries `mesh.geometricD()` and refines only the two active directions using OpenFOAM's directional refinement machinery.

One selected parent is split in two directions, producing four children. Therefore one refinement iteration gives

$$N_{new} = N_{old} + 3\,N_{selected} .$$

The F3 localized detonation test selected 384 cells from a 1200-cell base mesh and produced

$$1200+3(384)=2352$$

cells exactly.

3.3 Geometry detection: `auto`, `slab`, `wedge`

Supported settings are:

```
geometry auto;
geometry slab;
geometry wedge;
```

`auto` examines boundary patch types:

- one or more `empty` patches and no wedge patches -> `slab`;
- at least two `wedge` patches and no empty patches -> `wedge`.

Mixed/ambiguous geometry is rejected rather than guessed.

The mesh must report exactly two geometric dimensions:

$$n_{geometricD}=2.$$

3.4 Field mapping

Topology changes are applied through the normal OpenFOAM `fvMesh` update/mapping path. Registered fields are mapped with the topology map; `planarRefiner` does not keep private copies of flow/species fields.

This design was essential for maintaining pressure, temperature, velocity, and all species fields across localized refinement.

3.5 Restart state: n_{refine}

The only persistent private state is the global refinement-iteration count:

```
<time>/polyMesh/planarRefinerState
```

It stores, among other metadata:

```
nRefinementIterations
```

On restart, all MPI ranks must observe the same iteration count. If a case has already consumed its configured refinement iterations, it will not refine again merely because the solver restarted.

3.6 MPI distribution

`planarRefiner` stores no cell-indexed private data. During native mesh redistribution, OpenFOAM owns rank-local mesh/field movement and calls:

```
planarRefiner::distribute(const polyDistributionMap&)
```

The callback is intentionally metadata-free apart from diagnostic logging.

4. Independent build

4.1 From the release root

Source OpenFOAM Foundation 14 and run:

```
./AllwmakeAMR
```

The detonation solver is not compiled by this command.

4.2 Directly from the source directory

```
cd src/planarFvMeshTopoChangers
./Allwmake
```

Both paths build:

```
$FOAM_USER_LIBBIN/libplanarFvMeshTopoChangers.so
```

The library links only against OpenFOAM mesh/topology/finite-volume libraries; it does not link against `libdetonationFluidSolver.so`.

5. Basic `dynamicMeshDict`

A localized slab example is:

```
FoamFile
{
    format  ascii;
    class   dictionary;
    object  dynamicMeshDict;
}

topoChanger
{
    type                    planarRefiner;
    libs                    ("libplanarFvMeshTopoChangers.so");

    geometry                slab;
    refineInterval          1;
    field                   T;
    lowerRefineLevel        800;
    upperRefineLevel        4900;
    maxCells                200000;
    maxRefinementIterations 1;
}
```

Parameter meanings:

Entry	Meaning
`type`	must be `planarRefiner`
`libs`	loads `libplanarFvMeshTopoChangers.so`
`geometry`	`auto`, `slab`, or `wedge`
`refineInterval`	attempt refinement every N time indices; must be >= 1
`field`	registered scalar field used for selection
`lowerRefineLevel`	inclusive lower selection threshold
`upperRefineLevel`	inclusive upper selection threshold
`maxCells`	global hard cap; refinement is skipped if the exact growth bound would exceed it
`maxRefinementIterations`	maximum number of successful planar refinement iterations; default 1

6. Slab/`empty` usage

6.1 Mesh requirement

A slab case must be a true 2-D OpenFOAM mesh with the inactive direction represented by `empty` patches. `planarRefiner` validates this rather than accepting a thin 3-D mesh as a substitute.

6.2 Uniform one-level smoke

To refine every cell once, choose a band that contains the full field range, for example:

```
field                   T;
lowerRefineLevel        0;
upperRefineLevel        10000;
maxRefinementIterations 1;
```

The F2 qualification started with 1200 cells and produced 4800 cells, with strict final `checkMesh` success.

6.3 Localized refinement

Localized directional refinement can create a small conformal transition region between coarse and refined cells. In the qualified detonation slab:

- base cells: 1200;
- selected cells: 384;
- final cells: 2352;
- transition polyhedra: 4;
- 7-face transition cells: 4;
- concave cells: 4;
- max non-orthogonality: 18.435 degrees;
- max skewness: 0.3333;
- max $\left|\sum_i Y_i - 1\right|$: 1.045×10^{-8}.

Therefore a localized planar case should be judged by topology/geometry quality and field conservation, not by requiring every transition cell to remain a perfect hexahedron.

7. Wedge/axisymmetric usage

7.1 Geometry

A wedge case must contain a valid pair of `wedge` constraint patches. Use:

```
geometry wedge;
```

or `geometry auto;` when the boundary is unambiguous.

`planarRefiner` identifies the inactive geometric direction from OpenFOAM's geometric-dimension information and refines the two active directions only. The normal OpenFOAM topology-change correction preserves the wedge constraint.

7.2 Qualified portability case

The F3 portability test used stock OpenFOAM 14 `incompressibleFluid` through `foamRun`:

- annular wedge geometry;
- 1600 initial cells;

- one uniform planar level;
- 6400 final cells;
- both wedge patches preserved;
- strict final `checkMesh`: `Mesh OK.`.

This demonstrates that `planarRefiner` is not detonation-specific.

8. Native load balancing

8.1 Distributor block

`planarRefiner` can coexist with OpenFOAM's native runtime load balancer:

```
distributor
{
    type                    loadBalancer;
    libs                    ("libfvMeshDistributors.so");
    redistributionInterval  2;
    maxImbalance            0;
    multiConstraint         false;
}
```

8.2 Runtime decomposition method

The load balancer reads the active `system/decomposeParDict`. For Scotch:

```
numberOfSubdomains 2;
method scotch;
```

If a deterministic `simple` initial decomposition is desired for testing, perform `decomposePar` with the simple dictionary first, then replace the active dictionary with the Scotch version before `foamRun` starts.

8.3 What to look for in logs

A genuinely exercised native load-balancing run should show evidence such as:

```
Selecting fvMeshDistributor loadBalancer
Selecting distributor scotch
Imbalance of load ...
Redistributing mesh
planarRefiner: received native mesh-distribution callback
```

9. Restart from a refined/distributed mesh

The F3 restart qualification used a two-rank case that:

1. started from a deterministic decomposition;
2. applied localized planar refinement;
3. performed native load balancing;
4. wrote the refined/distributed mesh and `planarRefinerState`;
5. restarted in place with `startFrom latestTime`;
6. restored refinement iteration `1/1`;
7. completed without a second refinement.

The continuous and restarted final geometries agreed to numerical roundoff, and the major-species NRMSE was below 10^{-9} in the qualification case.

10. Using `planarRefiner` with detonationFoam

Build both components:

```
./Allwmake
./AllwmakeAMR
```

Then add the `topoChanger` block to the detonation case `constant/dynamicMeshDict`. No changes to `applications/modules/detonationFluid` are required.

A typical detonation selection band is:

```
field             T;
lowerRefineLevel  800;
upperRefineLevel  4900;
```

This is only an example from the qualified NH3/O2 case. Selection thresholds should be chosen for the physics and field scales of the new case.

11. Using `planarRefiner` without detonationFoam

Only the AMR library must be built:

```
./AllwmakeAMR
```

A different solver can then load the library through `constant/dynamicMeshDict`. The solver must participate correctly in OpenFOAM's normal dynamic-mesh/topology-change lifecycle and must have any topology-change flux-correction settings it requires.

For example, the F3 `incompressibleFluid` wedge portability test required a `pcorr` solver entry and `correctPhi yes` because stock incompressible dynamic-mesh flux correction uses `pcorr` after topology change. That requirement belongs to `incompressibleFluid`, not to `planarRefiner` itself.

12. Limitations and safe interpretation

Current release limitations are:

- OpenFOAM Foundation 14 only;
- true 2-D meshes only (`nGeometricD()==2`);
- supported geometry is slab/`empty` or wedge/axisymmetric;
- scalar-field band selection only;
- refinement-only; no automatic unrefinement/coarsening;
- the current exact cell-growth guard assumes the one-pass planar four-child split;
- localized transition cells can be non-hex polyhedra and should be evaluated with `checkMesh` and conservation diagnostics.

The library has been qualified for MPI2/native redistribution and restart, but long endurance and formal large-scale parallel testing are deferred post-release.

13. Troubleshooting

13.1 `planarRefiner requires a 2-D mesh`

The mesh is being recognized as three-dimensional. Check `empty`/`wedge` constraint patches and mesh construction. A one-cell-thick 3-D mesh is not automatically a true 2-D mesh.

13.2 Geometry mismatch

If `geometry slab;` is requested but the boundary contains wedge patches, or vice versa, the component exits intentionally. Use the correct constraint patches or `geometry auto;` for an unambiguous mesh.

13.3 Refinement is skipped

Possible reasons include:

- the current time index is not a `refineInterval` multiple;
- the configured `maxRefinementIterations` has already been reached;
- no cells are inside the selected scalar band;
- the exact projected growth $N_{old} + 3\,N_{selected}$ would exceed `maxCells`.

13.4 Restart refines twice

Check that the restart time contains:

`polyMesh/planarRefinerState`

and that the case is restarting from the intended written time. The log should report the restored refinement iteration.

13.5 MPI redistribution uses `simple` instead of Scotch

The active `system/decomposeParDict` still says `method simple;`. Change it to `method scotch;` before runtime load-balancer construction.

14. Source layout

```
src/planarFvMeshTopoChangers/
    Allwmake
    Make/files
    Make/options
    README.md
    planarRefiner/
        planarRefiner.H
        planarRefiner.C
        planarMultiDirRefinement.H
        planarMultiDirRefinement.C
        planarRefinementIterator.H
        planarRefinementIterator.C
```

The directional refinement implementation is based on OpenFOAM Foundation topology-change machinery and remains licensed under the GPL terms carried by the source files and release `LICENSE`.